

Constraint Acquisition of Configuration Knowledge Bases with Maximum Satisfiable Subsets

Alexander Felfernig^{1*}, Viet-Man Le^{1,2*}, Sebastian Lubos¹ and
Thi Ngoc Trang Tran^{1,3}

¹Institute of Software Engineering and Artificial Intelligence, Graz
University of Technology, Inffeldgasse 16b, Graz, 8010, Austria.

²University of Economics, Hue University, Hue, Vietnam.

³School of Hospitality and Tourism, Hue University, Hue, Vietnam.

*Corresponding author(s). E-mail(s): alexander.felfernig@tugraz.at;
v.m.le@tugraz.at;

Contributing authors: sebastian.lubos@tugraz.at; trang.tran@tugraz.at;

Abstract

Constraint acquisition techniques help to accelerate the process of generating configuration knowledge bases for software product lines. In such scenarios, constraints are derived on the basis of examples which are user-defined or system-generated and classified by agents (e.g., users, domain experts, and oracles) as valid (positive) or invalid (negative). In this paper, we introduce a new constraint acquisition approach that uses maximum satisfiable subsets (MSS) of constraint biases as a basis for generating constraint theories. Our approach generalizes existing constraint acquisition approaches by allowing partial example definitions helpful in situations where user-specified and system-generated examples should be combined. We demonstrate the applicability of our approach with real-world feature model knowledge bases and provide a theoretical analysis of the algorithm's time complexity, correctness, and completeness.

Keywords: keyword1, Keyword2, Keyword3, Keyword4

1 Introduction

The size and complexity of real-world knowledge bases make their development and maintenance particularly challenging which creates a demand for effective *constraint*

acquisition methods [1, 2]. Constraint acquisition typically leverages example data gathered from different agents such as users, domain experts, and external systems (oracles) to derive a constraint theory that accepts a set of positive examples while rejecting a set of negative ones [3–5]. Constraint acquisition can be viewed as an interaction between an agent and a learner. The agent knows the target theory through examples of valid and invalid solutions but does not know how to model it as a constraint network. The learner aims to induce a constraint network that accepts the provided positive examples and rejects the provided negative examples [1, 6].

Passive learning generates constraints from predefined positive and negative examples, which are often derived from completed configurations or user-provided data. This approach is characterized by the absence of direct user interaction during the learning process [7]. Instead, the learning algorithm operates solely on given examples and predefined constraints, which are typically formulated based on prior knowledge. Examples serve as the foundation for constructing a constraint theory. In passive learning, the learning process is largely independent of real-time user feedback or active exploration of the problem space. The success of the learning process heavily depends on the quality and diversity of the provided examples. Despite its reliance on static examples, passive learning can be effective in scenarios where acquiring new data or engaging users interactively is impractical or costly.

Active learning is based on the idea of generating examples that users classify as solutions or non-solutions through an interactive process [8, 9]. Users evaluate such examples by answering *membership queries* [10]. One of the key advantages of this approach is that users do not need to generate examples on their own. Alternatively, also legacy systems (as oracles) can be applied to evaluate containment queries [11]. Active learning employs a query-driven approach, where the user’s responses guide the generation of new examples [12]. This approach can result in a resource-intensive amount of queries [8, 9]. To enhance efficiency, different techniques can be applied, for example, probabilistic classification can predict constraint membership which streamlines query generation [13].

Further constraint acquisition approaches include *association rule mining* [14] which focuses on the extraction of representative constraints (e.g., filter constraints and compatibility relationships) describing the relationships between user requirements and corresponding product properties. Typically, there is no guarantee that the generated rules reflect the domain knowledge. Therefore, follow-up user-driven quality assurance of the generated constraints is needed. [15] focus on the generation of *decision trees* where related rules support a classification between high-quality and low-quality parameter settings (configurations) for video generation. Decision trees are also applied in the context of learning appropriate document templates for specific writing tasks (see [16]).

In this paper, we propose a novel approach to passive constraint acquisition which determines a constraint theory based on a maximum satisfiable subset (MSS) of a defined constraint bias. The main contributions of our paper are the following: (1) we introduce a divide-and-conquer based algorithm for efficient constraint acquisition, which identifies maximal satisfiable constraint subsets (MSS) that are used as a basis for generating constraint theories; (2) we demonstrate the algorithm’s applicability

with real-world configuration datasets; (3) we provide a theoretical analysis of the algorithm’s time complexity, correctness, and completeness.

The remainder of this paper is organized as follows. In Section *Preliminaries and Working Example*, we introduce a constraint satisfaction problem (configuration of software installations) that will be used as a working example in this paper. In Section *MMS-based Constraint Acquisition*, we sketch the overall idea of our constraint acquisition approach and provide details of the algorithm. An evaluation of the algorithm is presented in Section *Analysis and Evaluation*.

2 Preliminaries and Working Example

Example Scenario

We now introduce a scenario where the goal is to construct a configuration knowledge base that helps to avoid configurations that lead to inefficient (or erroneous) software installations on a computer. In this scenario, a constraint-based configurator represents the *solution space* (the allowed software configurations) and the underlying platform (hardware and operating system) represents the *problem space* [17]. We regard a configuration (a software installation) as a *negative example* ($e^- \in E^-$), if the corresponding installation leads to low performance or failure. Otherwise, an installation is regarded as a positive example ($e^+ \in E^+$). Estimating whether a configuration is a positive or a negative example is the task of an oracle.

An example can be automatically generated and classified as positive or negative by evaluating it on a corresponding platform (the oracle). This setting is a basic form of passive learning where examples are assumed to be available prior to the start of the learning process. Our approach does not rely on direct user interaction, which can be impractical due to time constraints and limited domain expertise. Even technical experts are generally unable to accurately assess the efficiency of arbitrary software configurations. When an oracle is available, user feedback can therefore be replaced by automated evaluation which enables constraint acquisition to be performed without manual intervention.

Working Example

For demonstration purposes, we introduce a simplified example from the domain of software configuration. The objective is to learn a constraint theory that captures which software components may or must not be combined in order to prevent inefficiency or failures during runtime. For our example constraint acquisition task, we assume the definition of three Boolean variables $x_i \in V$ (with corresponding domain definitions ($\text{domain}(x_i)$) as depicted in Table 1. These variables indicate the inclusion or exclusion of a software component in or from a specific configuration. Following the example in Table 1, we introduce a definition of a *vocabulary* [1] (see Definition 1).

Definition 1 (Vocabulary) The combination (V, D) of variables $V = \{x_1..x_k\}$ and domain definitions $D = \{\text{domain}(x_1).. \text{domain}(x_k)\}$ is denoted as *vocabulary*.

Table 1: CSP variables and domains (vocabulary) of an example software configuration task.

$x_i \in V$	$\text{domain}(x_i)$	software component
id	$\{0, 1\}$	<i>development environment</i>
db	$\{0, 1\}$	<i>database system</i>
ga	$\{0, 1\}$	<i>high-end computer game</i>

A learner starts with such a vocabulary definition which is the basis for the learning of a constraint theory (Definition 2).

Definition 2 (Constraint Theory) A finite set of constraints $C = \{c_1..c_n\}$ over a vocabulary V is a *constraint theory*.

Definition 3 (Target Constraint Theory) Given a vocabulary (V, D) , a *target constraint theory* C_τ is the (unknown) constraint theory over V that exactly characterizes the intended solutions (the ground-truth set of constraints that a learner aims to identify).

An example target constraint theory C_τ , i.e., the desired or ideal constraints for our learning example, is shown in Table 2. It is important to note that, in practice, such constraints are often not available and are thus unknown to the learner. The goal of constraint acquisition is to identify – or at least approximate – the target constraints as accurately as possible.

Table 2: Example target constraint theory C_τ .

$ct_i \in C_\tau$	constraint	explanation
ct_1	$id \rightarrow db$	<i>id needs db</i>
ct_2	$id \not\rightarrow ga$	<i>low performance setting</i>

Table 3: Constraints of the bias B ($\rightarrow$ and $\wedge$ denote *logical implication* and *logical and*, whereas $\not\rightarrow$ denotes the corresponding negation). This scheme represents all possible variable/operator combinations. In our example, we have 18 different generated constraints $B = \{c_1..c_{18}\}$.

B	db	id	ga
db	–	$\rightarrow_{(c1)} \wedge_{(c2)} \not\rightarrow_{(c3)}$	$\rightarrow_{(c4)} \wedge_{(c5)} \not\rightarrow_{(c6)}$
id	$\rightarrow_{(c7)} \wedge_{(c8)} \not\rightarrow_{(c9)}$	–	$\rightarrow_{(c10)} \wedge_{(c11)} \not\rightarrow_{(c12)}$
ga	$\rightarrow_{(c13)} \wedge_{(c14)} \not\rightarrow_{(c15)}$	$\rightarrow_{(c16)} \wedge_{(c17)} \not\rightarrow_{(c18)}$	–

Training Set

The examples part of the training set $E = E^+ \cup E^-$ are not limited to (system-generated) complete variable value assignments evaluated by oracles, but can take the form of user-provided *partial examples* where not all variable values are explicitly specified (see the entries in Table 4). In our software configuration scenario, this is the case if *technical experts* have some well-working (positive) and non-performing or even error-inducing (negative) examples in mind and want to specify these as input for the learning process. Each positive example needs to be consistent with the generated constraint theory whereas each negative one needs to be rejected. For negative examples, we require *subset minimality*, i.e., each generated negative example is a minimal conflict with regard to the underlying target theory. For example, e_1^- in Table 4 is such a conflict with regard to the example target constraint theory defined in Table 2.

Table 4: Positive ($e^+ \in E^+$) and negative examples ($e^- \in E^-$) specify intended semantics to be taken into account by the learner. Our approach supports both, complete and partial variable value assignments.

test case	constraint
e_1^+	$\neg id \wedge ga$
e_2^+	$id \wedge db \wedge \neg ga$
e_1^-	$id \wedge \neg db$

Background Knowledge

For our constraint learning scenario, we assume that it is already known (expert knowledge) that the high-end game (*ga*) requires the inclusion of a database (*db*). Such a constraint is assumed to exist a-priori and is therefore not subject to learning – we denote this as *background knowledge BG* (see Table 5).

Table 5: Example constraint representing the background knowledge $BG=\{cb_1\}$.

$cb_i \in BG$	constraint
cb_1	$ga \rightarrow db$

Constraint Language and Bias

Constraint theories as well as examples are defined on the basis of a *constraint language* L (see Definition 4). A *constraint bias* expressed in L [6] is a set of constraints that can be used for learning a constraint theory (see Definition 5).

Definition 4 (Constraint Language) Given a vocabulary (V, D) , a *constraint language* L is a formal language that (implicitly) defines the admissible constraints over (V, D) .

Definition 5 (Constraint Bias) Given a vocabulary (V, D) and a constraint language L , a *constraint bias* B is a finite set of constraints expressible in L that defines the space of candidate constraints for a constraint theory.

We assume a set of candidate constraints $B = \{c_1, \dots, c_k\}$, which defines the space of constraints that may be used to construct a constraint theory (knowledge base) KB . In our illustrative bias, we permit binary operators over the variables db , id , and ga , namely conjunction $\wedge$, negated conjunction $\neg\wedge$, and implication $\rightarrow$. Accordingly, the language of admissible constraints is defined as

$$L = \{x_i \circ x_j \mid i \neq j, \circ \in \{\wedge, \neg\wedge, \rightarrow\}\}.$$

Note that, in the general case, we do not impose any restrictions on the structural form of constraints. The specific constraints included in B for our example are listed in Table 3.

Based on previous definitions, we introduce the concept of a *constraint acquisition problem* (see Definition 6).

Definition 6 (Constraint Acquisition Problem) Given a constraint bias B , a background knowledge BG , and a training set $E = E^+ \cup E^-$, the task of finding a constraint theory C accepting all $e^+ \in E^+$ and rejecting all $e^- \in E^-$ is denoted as *constraint acquisition problem*.

3 MSS-based Constraint Acquisition

The idea of the CONGEN algorithm is to determine a *constraint theory* KB which is based on a maximum satisfiable subset (MSS) of a predefined bias B which supports all positive and rejects all negative examples. The idea is to define a bias B and then to learn a constraint theory on the basis of a set of positive examples E^+ and a set of negative examples E^- specifying (un)intended properties of KB . Before introducing our constraint acquisition approach, we sketch the handling of positive and negative examples. The constraints in Table 3 represent a complete enumeration of constraints that can be used in the context of our working example. For example, for specifying the relationship between the variables db and id , our learning algorithm can use the constraints $\rightarrow$ (*logical implication*), $\wedge$ (*logical and*), and $\neg$ (*negated and*). We now analyze which constraints will remain in B when taking into account the examples shown in Table 4. The corresponding constraint set B' is depicted in Table 6.

Table 6: Constraints $B' \subseteq B$ such that: $\forall e^+ \in E^+ : \text{consistent}(B' \cup \{e^+\} \cup NE \cup BG)$.

B'	db	id	ga
db	—	—	—
id	$\rightarrow_{(c7)}$	—	$\neg_{(c12)}$
ga	$\rightarrow_{(c13)}$	$\neg_{(c18)}$	—

To be able to take into account the positive examples e_1^+ and e_2^+ , all bias (B) constraints from Table 3 need to be removed which are inconsistent with some of these examples, the background knowledge (BG), and the negated negative examples (NE). For example, the bias constraint $db \rightarrow id$ directly contradicts with NE . Therefore, it has to be deleted from bias B . The constraints $B' \subseteq B$ which have not been deleted from the bias satisfy $\forall e^+ \in E^+ : \text{consistent}(\{e^+\} \cup B' \cup NE \cup BG)$.

Furthermore, we have to assure that all negative examples are inconsistent with the generated knowledge base KB . In our algorithm, this is achieved by including the negated negative examples (NE) in the resulting knowledge base. Finally, we are able to delete redundant constraints which results in the final constraint theory $\{c_7, c_{12}, c_{13}\}$.

ConGen (Algorithm 1)

NE is a set of constraints such that: if $e_i^- \in E^-$ then $\neg e_i^- \in NE$. In this context, we assume *subset minimality* of negative examples, for instance, if $id \wedge \neg db$ is a negative example (see Table 4), then omitting just one of those elements would resolve the underlying conflict. Such a minimality of negative examples can be achieved by asking users or external systems (oracles) to answer containment queries for the purpose of identifying minimal conflict sets [6]. The task of identifying one conflict set per negative example and generating NE thereof is supported by the function GENERATENE.¹ If we extend $e_1^- : id \wedge \neg db$ to $e_{1ext}^- : id \wedge \neg db \wedge ga$, we can use QUICKXPLAIN to filter out a preferred minimal conflict set $id \wedge \neg db$. In this context, an oracle can be used to answer containment queries. The function *IsConsistent* checks the condition $\forall e_i^+ \in E^+ : \text{consistent}(\{e_i^+\} \cup NE \cup BG)$ to assure that positive examples, negative examples, and given background knowledge BG do not interfere with each other. BG includes constraints known to be correct, for example, if the feature hierarchy of a configuration model is known and stable, corresponding constraints could be regarded as background knowledge. If *IsConsistent* returns *true*, Algorithm 2 (ACQMSS) can be activated.

AcqMss (Algorithm 2)

The goal is to identify a maximum satisfiable subset $B' \subseteq B$ (the constraint bias) such that $\forall e_i^+ \in E^+ : \text{consistent}(\{e_i^+\} \cup B' \cup BG \cup NE)$. ACQMSS follows a divide-and-conquer based approach which is based on the following idea: if a constraint set B is inconsistent, ACQMSS determines a maximum satisfiable subset (MSS) which accepts all positive examples defined in E^+ . If $B = \{c_1..c_n\}$ is inconsistent (no solution could be identified), it is divided into the parts $B_1 = \{c_1..c_k\}$ and $B_2 = \{c_{k+1}..c_n\}$. If $\forall e_i^+ \in E^+ : \text{consistent}(\{e_i^+\} \cup B_1 \cup BG \cup NE)$, it is returned as a part of the constraint theory B' . ACQMSS results in a maximum satisfiable set $B' \subseteq B$ which is guaranteed to accept all positive examples.

Reduce (Algorithm 3)

In a final step, NE is included into the learned constraint theory KB and made irredundant. The idea is to check each individual constraint in KB for redundancy: if

¹GENERATENE(E^-) activates QUICKXPLAIN [18] once per negative example $e^- \in E^-$.

Algorithm 1 CONGEN $(E^+, E^-, B, BG) : KB$

```
1:  $NE \leftarrow \text{GENERATEN}(E^-)$ 
2:  $B' \leftarrow \emptyset$ 
3: if IsConsistent( $E^+, NE, BG$ ) then
4:    $B' \leftarrow \text{ACQMSS}(\emptyset, B, NE, E^+, BG)$ 
5: else
6:   print "examples inconsistent"
7:   return ( $\emptyset$ )
8: end if
9: return ( $\text{REDUCE}(B', NE, BG)$ )
```

Algorithm 2 ACQMSS $(\delta, B=\{c_1..c_n\}, NE, E^+, BG) : B'$

```
1: if  $\delta \neq \emptyset$  then
2:   if IsConsistent( $E^+, NE, B, BG$ ) then
3:     return ( $B$ )
4:   end if
5: end if
6: if  $|B| = 1$  then
7:   return ( $\emptyset$ )
8: end if
9:  $k = \lfloor \frac{|B|}{2} \rfloor$ 
10:  $B_1 = \{c_1..c_k\}$ 
11:  $B_2 = \{c_{k+1}..c_n\}$ 
12:  $B'_\beta \leftarrow \text{ACQMSS}(B_1, B_1, NE, E^+, BG)$ 
13:  $B'_\alpha \leftarrow \text{ACQMSS}(B_1 - B'_\beta, B_2, NE, E^+, BG \cup B'_\beta)$ 
14: return ( $B'_\alpha \cup B'_\beta$ )
```

inconsistent($BG \cup B' - \{c_i\} \cup \{\neg c_i\}$), c_i can be regarded as redundant. This corresponds to the check for logical entailment: $BG \cup B' - \{c_i\} \cup \{\neg c_i\} \models c_i$.

Algorithm 3 REDUCE $(B', NE, BG) : KB$

```
1:  $KB \leftarrow B' \cup NE$ 
2: for  $c_i \in KB$  do
3:   if IsInconsistent( $BG \cup (KB - \{c_i\}) \cup \{\neg c_i\}$ ) then
4:      $KB \leftarrow KB - \{c_i\}$ 
5:   end if
6: end for
7: return ( $KB$ )
```

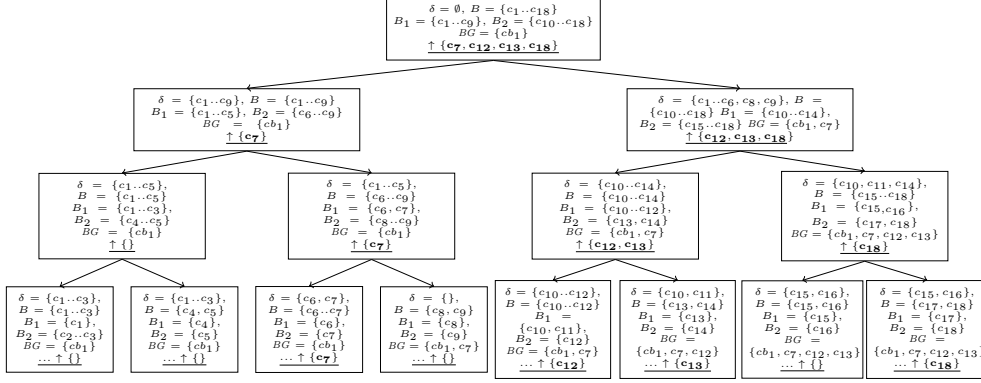


Fig. 1: ACQMSS execution trace (limited to search depth 3) for $C = \{c_1..c_{18}\}$, $NE = \{\neg(id \wedge \neg db)\}$, $E^+ = \{e_1^+, e_2^+\}$ and $BG = \{ga \rightarrow db\}$ ($\uparrow$ denotes the return values of different ACQMSS incarnations). The resulting maximal satisfiable subset $\{c_7, c_{12}, c_{13}, c_{18}\}$ satisfies the property: $\forall e^+ \in E^+ : \text{consistent}(\{e^+ \cup NE \cup BG\})$.

Application of ConGen

We now start a short walkthrough of CONGEN. First, we have to construct NE which is $NE = \{\neg(id \wedge \neg db)\}$. Since $IsConsistent(E^+, NE, BG)$, we are able to activate ACQMSS. For our example, we assume the bias depicted in Table 3. CONGEN returns a constraint theory $KB = \{c_7, c_{12}, c_{13}\}$. A corresponding ACQMSS execution trace is shown in Figure 1.

4 Analysis and Evaluation

CONGEN and ACQMSS have the following properties.

Time Complexity

The *worst-case* complexity of ACQMSS in terms of the number of needed consistency checks for determining one maximum satisfiable subset B' is $2\gamma \times \log_2(\frac{n}{\gamma}) + 2\gamma$ where γ is the *number of elements deleted from B*, n is the *number of constraints in B*, and 2γ represents the branching factor and the number of leaf-node consistency checks. In this context, we interpret the checking of all positive examples of E^+ for consistency with $NE \cup B \cup BG$ as *one consistency check*. The corresponding *best-case* complexity is $\log_2(\frac{n}{\gamma}) + 2\gamma$. In the worst case, each element to be excluded from B is located in a different path of the search tree. The factor $\log_2(\frac{n}{\gamma})$ represents the depth of a path of the ACQMSS search tree. In the best case, each to be excluded element is located in the same path of the search tree.

If n is the number of constraints in B removed by ACQMSS, QUICKXPLAIN [18] requires $\log_2(\frac{n}{\pi}) + 2\pi$ consistency checks in the best case and $2\pi \times \log_2(\frac{n}{\pi}) + 2\pi$ in the worst case (π is the assumed conflict set size). With an increasing number of conflicts in B , traditional HSDAG-based conflict resolution [19] performance deteriorates (for details, see our performance evaluation).

Theorem 1 (Correctness of ACQMSS) *Let $B' \subseteq B$ (the constraint bias) be the set of constraints returned by ACQMSS – then B' accepts all positive examples.*

Proof Sketch ACQMSS is only activated, if the following property is fulfilled: $\forall e^+ \in E^+ : \text{consistent}(\{e^+\} \cup NE \cup BG)$. If fulfilled, ACQMSS incrementally aggregates the sets B'_β into BG which fulfill: $\forall e^+ \in E^+ : \text{consistent}(\{e^+\} \cup NE \cup BG)$. This way, all $e^+ \in E^+$ are accepted. $\square$

Theorem 2 (Completeness of CONGEN) *If $\forall e^+ \in E^+ : \text{consistent}(\{e^+\} \cup NE \cup BG)$, CONGEN will return $B' \subseteq B$.*

Proof Sketch If $\forall e^+ \in E^+ : \text{consistent}(\{e^+\} \cup NE \cup BG)$, CONGEN activates ACQMSS which returns $B' \subseteq B$. In the worst case, all constraints are removed, i.e., $B' = \emptyset$. $\square$

Remark 1 If $B' = \emptyset$, CONGEN will only return the negated negative examples NE derived from E^- .

Corollary 1 *ACQMSS fails to compute a bias reduction B' if and only if the following condition is violated: $\exists e^+ \in E^+ : \text{inconsistent}(\{e^+\} \cup NE \cup BG)$.*

Evaluation Dataset

Positive (and negative) examples can stem, for example, from (non-)solutions (e.g., configurations) generated by a knowledge base that should be substituted by a different one (e.g., within the scope of a technical replacement). Furthermore, examples can stem from inconsistent user requirements defined on such a knowledge base. Positive and negative examples could also be interactively collected from users which, however, could become a rather tedious process. Beyond that, positive and negative examples can be generated on the basis of so-called oracle which are external systems providing feedback on the feasibility of pre-generated configurations. This is a form of configuration space learning [20] with the goal to exploit examples for generating a model representation of the configuration space. In such a scenario, a solution generator could create different configurations (examples) which are then evaluated by the software components (i.e., does the generated configuration lead to errors or inefficiencies?). If the configuration can be regarded as a positive example, nothing else needs to be done. In our evaluation, we apply real-world configuration knowledge bases [21] for the purpose of (oracle-based) query answering. These knowledge bases are represented in terms of feature hierarchies and corresponding constraints between individual features (see, e.g., Table 2). For evaluation purposes, we kept constraints specifying the feature hierarchy in the background knowledge (BG) and constraints specifying component requirements and incompatibilities as elements of the constraint bias B .

Sampling Methods

For our evaluation, we use feature model knowledge bases from [21] as an oracle, i.e., the knowledge base is used to generate positive (solutions) and negative examples (non-solutions). For example generation, we used the following three feature selection approaches which are frequently applied in testing scenarios [20]. First, we use *random sampling* (RS) with the number of examples equal to n , $2n$, $3n$, and m where n is the number of features (variables) and m is the smallest number of valid configurations including each pair of features. Such limits are often used in scenarios where the answering of a containment query can become effortful. Second, *n-wise coverage* (n-COV) represents a set of examples (configurations) that satisfies related coverage heuristics, for example, *2-wise coverage* (2-COV) results in a set of configurations where each combination of two features is included at least once. Finally, the *feature frequency* (FF) heuristics focuses on generating examples in such a way that each feature is at least once included and excluded in an example. This holds for both, positive as well as negative examples. For each generated test case, an oracle (in our evaluation settings, we apply real-world knowledge bases [21] to simulate oracles) provides feedback in terms of classifying a generated example e_η as a positive or negative.

Evaluation of Generated Knowledge Bases

In typical supervised machine learning scenarios, the set of examples is organized on the basis of a n -fold cross validation where the whole test set is divided into n subsets of similar sizes. In each validation iteration, one of the subsets is used as test set and the remaining ones are used as training sets. This way, each generated knowledge base (constraint theory) can be tested in terms of *accuracy* (see Formula 1).

$$accuracy(KB) = \frac{TP + TN}{TP + TN + FP + FN} \quad (1)$$

Accuracy is measured on the basis of *true positives* (TP) (number of positive examples accepted by KB), *true negatives* (TN) (number of rejected negative examples), *false positives* (FP) (number of accepted negative examples), and *false negatives* (FN) (number of rejected positive examples).

This way, we are able to take into account the aspect of knowledge generalizability into constraint acquisition approaches. Table 7 depicts a comparison of the average number of ACQMSS consistency checks needed for determining one maximal satisfiable subset (MSS) for pre-defined examples (positive and negative) and a pre-defined bias B .

Table 7: ACQMSS *#consistency checks* and *runtime* (in *msec*) for different real-world configuration knowledge bases. For simulating test case generation, these knowledge bases have been applied as an oracle.

$ E^+ $	$ E^- $	KB_1	KB_2	KB_3	KB_4	KB_5
TBD	TBD	TBD	TBD	TBD	TBD	TBD
...	...	...	...	...	...	...

The major task of ACQMSS is to resolve conflicts in the constraint bias B with regard to the defined positive examples. Table 8 shows a performance comparison between ACQMSS and HSDAG based approaches [9, 19].

Table 8: Comparison of ACQMSS *#consistency checks* with a HSDAG-based approach to conflict resolution in the constraint bias B .

$ E^+ $	$ E^- $	KB_1	KB_2	KB_3	KB_4	KB_5
TBD	TBD	val1/val2	TBD	TBD	TBD	TBD
...	...	...	...	...	...	...

Table 9 depicts a comparison of ACQMSS of the average accuracy achieved by CONGEN for different test suite sizes (test suites generated with random sampling (RS)).

Table 9: *Accuracy* of generated example knowledge bases derived on the basis of test cases generated with *random sampling* (RS).

$ E^+ $	$ E^- $	KB_1	KB_2	KB_3	KB_4	KB_5
TBD	TBD	TBD	TBD	TBD	TBD	TBD
...	...	...	...	...	...	...

Table 10 depicts a comparison of ACQMSS of the average accuracy achieved by CONGEN for different test suite sizes (test suites generated with 2-wise coverage – 2-COV).

Table 10: *Accuracy* of generated example knowledge bases derived on the basis of test cases generated with *2-wise coverage* (2-COV).

$ E^+ $	$ E^- $	KB_1	KB_2	KB_3	KB_4	KB_5
TBD	TBD	TBD	TBD	TBD	TBD	TBD
...	...	...	...	...	...	...

Table 11 depicts a comparison of ACQMSS of the average accuracy achieved by CONGEN for different test suite sizes (test suites generated with feature frequency – FF).

Table 11: *Accuracy* of generated example knowledge bases derived on the basis of test cases generated with *feature frequency* (FF).

$ E^+ $	$ E^- $	KB_1	KB_2	KB_3	KB_4	KB_5
TBD	TBD	TBD	TBD	TBD	TBD	TBD
...	...	...	...	...	...	...

Table 12 depicts a comparison of the average accuracy achieved by iterative (active) learning approaches for different test suite sizes (test suites generated with [21]). With *iterative learning* we refer to learning approaches where individual test cases are generated in an interactive fashion and in each iteration the already learned constraint theory is taken into account when generating new test cases. Following [8], the idea is generate new test cases that are (1) consistent with the constraint in the already learned constraint theory and (2) inconsistent with at least one constraint still part of the constraint bias.

Table 12: Accuracy of generated example knowledge bases derived on the basis of test cases generated with the best-performing sampling approach (i.e., [TBD]) compared with iterative approaches.

$ E^+ $	$ E^- $	KB_1	KB_2	KB_3	KB_4	KB_5
TBD	TBD	TBD	TBD	TBD	TBD	TBD
...	...	...	...	...	...	...

5 Conclusion

In this paper, we have introduced a novel approach to constraint acquisition based on the identification of maximal satisfiable subsets of constraint biases. Our approach extends existing constraint acquisition methods by supporting partial examples which helps to reduce the burden of example specification for domain experts. In addition, we presented an application scenario that integrates oracles to support the task of answering containment queries which relieves domain experts from this responsibility. The proposed approach was successfully evaluated using real-world configuration knowledge bases, which were employed to simulate an oracle capable of answering containment queries.

Supplementary information. If your article has accompanying supplementary file/s please state so here.

Authors reporting data from electrophoretic gels and blots should supply the full unprocessed scans for key as part of their Supplementary information. This may be requested by the editorial team/s if it is missing.

Please refer to Journal-level guidance for any specific requirements.

Acknowledgements. Acknowledgements are not compulsory. Where included they should be brief. Grant or contribution numbers may be acknowledged.

Please refer to Journal-level guidance for any specific requirements.

Declarations

Some journals require declarations to be submitted in a standardised format. Please check the Instructions for Authors of the journal to which you are submitting to see if

you need to complete this section. If yes, your manuscript must contain the following sections under the heading ‘Declarations’:

- Funding
- Conflict of interest/Competing interests (check journal-specific guidelines for which heading to use)
- Ethics approval and consent to participate
- Consent for publication
- Data availability
- Materials availability
- Code availability
- Author contribution

If any of the sections are not relevant to your manuscript, please include the heading and write ‘Not applicable’ for that section.

Appendix A Section title of first appendix

An appendix contains supplementary information that is not an essential part of the text itself but which may be helpful in providing a more comprehensive understanding of the research problem or it is information that is too cumbersome to be included in the body of the paper.

References

- [1] Bessiere, C., Koriche, F., Lazaar, N., O’Sullivan, B.: Constraint acquisition. *Artificial Intelligence* **244**, 315–342 (2017) <https://doi.org/j.artint.2015.08.001>
- [2] Freuder, E., O’Sullivan, B.: Grand challenges for constraint programming. *Constraints* **19**, 150–162 (2014) <https://doi.org/10.1007/s10601-013-9155-1>
- [3] O’Connell, S., O’Sullivan, B., Freuder, E.C.: Timid acquisition of constraint satisfaction problems. In: *ACM Symposium on Applied Computing. SAC ’05*, pp. 404–408. ACM, New York, NY, USA (2005). <https://doi.org/10.1145/1066677.1066770>
- [4] Prestwich, S., Freuder, E., O’Sullivan, B., Browne, D.: Classifier-based constraint acquisition. *Annals of Mathematics and AI* **89**(7), 655–674 (2021) <https://doi.org/10.1007/s10472-021-09736-4>
- [5] Raedt, L.D., Passerini, A., Teso, S.: Learning constraints from examples. In: *32nd AAAI Conference on Artificial Intelligence. AAAI’18/IAAI’18/EAAI’18*, pp. 7965–7970. AAAI Press, ??? (2018)
- [6] Bessiere, C., Carbonnel, C., Dries, A., Hebrard, E., Katsirelos, G., Lazaar, N., Narodytska, N., Quimper, C., Stergiou, K., Tsouros, D.C., Walsh, T.: Learning

- constraints through partial queries. *Artif. Intell.* **319**, 103896 (2023) <https://doi.org/10.1016/J.ARTINT.2023.103896>
- [7] Bessiere, C., Coletta, R., Koriche, F., O’Sullivan, B.: Acquiring Constraint Networks Using a SAT-based Version Space Algorithm. In: 21st National Conf. on Artificial Intelligence and the 18th Innovative Applications of Artificial Intelligence Conf., pp. 1565–1568. AAAI Press, ??? (2006). <http://www.aaai.org/Library/AAAI/2006/aaai06-251.php>
 - [8] Bessiere, C., Coletta, R., Hebrard, E., Katsirelos, G., Lazaar, N., Narodytska, N., Quimper, C., Walsh, T.: Constraint acquisition via partial queries. *IJCAI ’13*, pp. 475–481. AAAI Press, ??? (2013)
 - [9] Shchekotykhin, K., Friedrich, G.: Argumentation based constraint acquisition. In: Wang, W., Kargupta, H., Ranka, S., Yu, P.S., Wu, X. (eds.) *ICDM 2009, The 9th IEEE International Conference on Data Mining, Miami, Florida, USA, 6-9 December 2009*, pp. 476–482. IEEE Computer Society, ??? (2009). <https://doi.org/10.1109/ICDM.2009.62> . <https://doi.org/10.1109/ICDM.2009.62>
 - [10] Angluin, D.: Queries and concept learning. *Machine Learning* **2**(4), 319–342 (1988) <https://doi.org/10.1023/A:1022821128753>
 - [11] Liem, C.C.S., Panichella, A.: Oracle Issues in Machine Learning and Where to Find Them. *ICSEW’20*, pp. 483–488. ACM, ??? (2020). <https://doi.org/10.1145/3387940.3391490> . <https://doi.org/10.1145/3387940.3391490>
 - [12] Bessiere, C., Coletta, R., O’Sullivan, B., Paulin, M.: Query-driven constraint acquisition. In: Veloso, M.M. (ed.) *20th International Joint Conference on Artificial Intelligence, Hyderabad, India, January 6-12, 2007*, pp. 50–55 (2007). <http://ijcai.org/Proceedings/07/Papers/006.pdf>
 - [13] Tsouros, D., Berden, S., Guns, T.: Learning to learn in interactive constraint acquisition. In: *arXiv* (2023). <https://arxiv.org/abs/2312.10795>
 - [14] Gibson, I., Yoke San, W., Huang, Y., Liu, H., Keong Ng, W., Lu, W., Song, B., Li, X.: Automating knowledge acquisition for constraint-based product configuration. *Journal of Manufacturing Technology Management* **19**(6), 744–754 (2008) <https://doi.org/10.1108/17410380810888120>
 - [15] Temple, P., Galindo, J.A., Acher, M., Jézéquel, J.-M.: Using machine learning to infer constraints for product lines. In: *20th Intl. Systems and Software Product Line Conference. SPLC ’16*, pp. 209–218. Association for Computing Machinery, New York, NY, USA (2016). <https://doi.org/10.1145/2934466.2934472> . <https://doi.org/10.1145/2934466.2934472>
 - [16] Acher, M., Temple, P., Jézéquel, J., Galindo, J., Martinez, J., Ziadi, T.: VARYLATEX: Learning Paper Variants That Meet Constraints. In: *12th Intl. Workshop*

- on Variability Modelling of Software-Intensive Systems, pp. 83–88. ACM, ??? (2018). <https://doi.org/10.1145/3168365.3168372>
- [17] Ochs, P., Pett, T., Schaefer, I.: Consistency is key: Can your product line realise what it models? In: ACM/IEEE 27th International Conference on Model Driven Engineering Languages and Systems, pp. 690–699. ACM, New York, NY, USA (2024). <https://doi.org/10.1145/3652620.3687812>
 - [18] Junker, U.: QUICKXPAIN: Preferred explanations and relaxations for over-constrained problems. In: 19th National Conference on Artificial Intelligence. AAAI’04, pp. 167–172. AAAI Press, ??? (2004)
 - [19] Reiter, R.: A theory of diagnosis from first principles. *Artificial Intelligence* **32**(1), 57–95 (1987) [https://doi.org/10.1016/0004-3702\(87\)90062-2](https://doi.org/10.1016/0004-3702(87)90062-2)
 - [20] Pereira, J.A., Acher, M., Martin, H., Jézéquel, J.-M., Botterweck, G., Ventresque, A.: Learning software configuration spaces: A systematic literature review. *Journal of Systems and Software* **182**, 111044 (2021) <https://doi.org/10.1016/j.jss.2021.111044>
 - [21] Heradio, R., Fernandez-Amoros, D., Galindo, J., Benavides, D., Batory, D.: Uniform and scalable sampling of highly configurable systems. *Empirical Softw. Eng.* **27**(2) (2022) <https://doi.org/10.1007/s10664-021-10102-5>